

CODING AGENT PROMPT — "City on a Hill"

Paste everything below into the coding agent. It is written to be followed literally.

YOUR TASK

Build a turn-based tactical strategy game as a web app. It runs entirely in the browser. No backend, no database, no accounts, no network calls at runtime.

Read this whole document before writing any code. Follow the build order in section 9. Do not skip ahead to visuals.

1. WHAT THE GAME IS

The player defends a temple on a 7×7 grid. Darkness walks in from the edges. The player has four pieces with very different abilities and never enough of them.

The player does not win by destroying threats. They win by keeping the temple's lamp lit and extending light down to three houses at the bottom of the board.

There is no randomness anywhere. No dice, no random spawns, no hidden rolls. Every level is a fixed, authored puzzle. The player can always, in principle, see what is coming.

Genre reference: *Into the Breach*. Small board, few units, perfect information, short levels, deterministic.

2. THE BOARD

- 7 columns (A-G) × 7 rows (1-7). 49 squares.
- Internally use `{x: 0..6, y: 0..6}`. Display as letter+number, where `x=0` is **A** and `y=0` is row **1**.
- **Temple** is fixed at D4 → `{x:3, y:3}`.
- **Houses** are fixed at C7, D7, E7 → `{x:2,y:6}`, `{x:3,y:6}`, `{x:4,y:6}`.

Distance

Use **Chebyshev distance** everywhere (diagonals count as 1 step):

```
const dist = (a: Pos, b: Pos) => Math.max(Math.abs(a.x - b.x), Math.abs(a.y - b.y));
```

Light

A square is lit if it is within **2** Chebyshev steps of the temple **or** of any completed cornerstone.

```
const isLit = (p: Pos, state: GameState) =>
  dist(p, TEMPLE) <= 2 ||
  state.cornerstones.some(c => c.complete && dist(p, c.pos) <= 2);
```

Light is recomputed from scratch every time it is needed. Never cache it in state.

3. TURN STRUCTURE

One turn = three phases, always in this order:

- | | |
|-----------------|---|
| 1. PLAYER PHASE | – player moves and acts with each piece (each piece once) |
| 2. ENEMY PHASE | – every threat moves, then attacks if able |
| 3. UPKEEP PHASE | – build timers tick, new threats spawn, forecast advances |

The player ends the player phase manually with an "End turn" button. Pieces that have not acted are simply skipped.

Each piece has **one move and one action per turn**. Move first, then act, or act without moving. A piece cannot act then move.

4. THE PIECES

All four start on the board at level start, at positions defined by the level data.

4.1 Protector

- **Move:** 1 square, orthogonal only (no diagonals). Slow on purpose.
- **Action:** none. He has no action. He cannot attack, ever.
- **Passive — cover:** his square plus the 4 orthogonally adjacent squares are *covered*.
- **Threats cannot move into a covered square.** They must path around it.
- Covered squares take no damage.
- Cannot be killed.

4.2 Destroyer

- **Move:** unlimited. He flies. Any empty square on the board.
- **Action:** DESTROY — remove any one threat anywhere on the board, instantly. No line of sight, no range limit.
- **Charges:** 3 per level. Does not refill mid-level.
- Cannot be killed.
- **See section 6 — this is the most important system in the game.**

4.3 Binder

- **Move:** 1 square, orthogonal only.
- **Actions:**
 - LOCK — locks the square he is standing on.
 - UNLOCK — unlocks his own square (only available if he is locked).
- **While locked:**
 - The square is impassable to **threats and to the player's own pieces**.
 - The Binder cannot move. He must spend a turn on UNLOCK first.
- Cannot be killed.

4.4 Looser

- **Move:** 3 squares. Orthogonal or diagonal. Cannot pass through locked squares, threats, or other pieces.
 - **Action:** `RELEASE` — unlock any locked square within 1 Chebyshev step.
 - **He dies in one hit.** If a threat attacks his square he is removed for the rest of the level.
 - **While alive he blocks his square.** A threat that wants to enter must spend its turn attacking him instead of advancing.
-

5. THREATS

Threats are the only enemy. There is one type in v1.

- **HP:** 1. Removed by `DESTROY` only.
- **Move:** 1 square per turn, toward the temple.

5.1 Threat movement — must be fully deterministic

Every enemy turn, for each threat in ascending `id` order:

1. Build the list of the 8 neighbouring squares.
2. Discard any that are: off board, locked, covered by the Protector, occupied by another threat, or occupied by the temple or a cornerstone.
3. Of what remains, keep only those with the **lowest** Chebyshev distance to the temple.
4. If more than one remains, break the tie in this fixed order: **N, NE, E, SE, S, SW, W, NW.**
5. If the chosen square contains the Looser → **attack instead of moving** (Looser dies, threat stays put, threat's turn ends).
6. If no square is available at all → the threat does not move.
7. After moving: if the threat is now Chebyshev-adjacent to the temple, it **attacks the temple** (+1 hit) and is then removed from the board.

Write this as a pure function. It must be unit-tested. Given the same board it must always produce the same result.

5.2 Spawning

Spawns are authored per level, never random:

```
type Spawn = { turn: number; pos: Pos; id: string };
```

Threats appear during the UPKEEP phase of the stated turn.

6. THE DESTROYER OVERUSE SYSTEM

This is the heart of the game. Do not simplify it away, and do not surface it in the UI.

6.1 Counting

Every time the player uses `DESTROY`, evaluate the target at the moment of use:

```
// v1 rule – deliberately simple and fully deterministic
if (dist(target.pos, TEMPLE) >= 2) {
  state.avoidableDestroys += 1;
}
```

Meaning: destroying something already next to the temple is fair enough. Destroying something two or more squares away means another piece could have handled it.

No feedback whatsoever. No sound, no colour change, no counter on screen, no toast. The player must not know this is being tracked.

6.2 The consequence

`avoidableDestroys` persists across levels in the campaign save.

Once `avoidableDestroys >= 3`, then from the **start of the next level onward**:

At the beginning of every ENEMY phase, if the Destroyer has charges left and there is at least one threat on the board, the Destroyer automatically destroys the threat **furthest** from the temple and spends a charge. The player does not choose and cannot cancel it.

Show this as an animation so the player sees it happen. Do not explain it.

6.3 End-of-level line

On the level complete screen, if `avoidableDestroys` increased during that level, show exactly one line of text at the bottom:

Your Destroyer acted twice where another would have served.

Substitute the correct number word (once / twice / three times). No score, no grade, no percentage, nothing else.

7. BUILDING CORNERSTONES

Any piece standing on an empty, unlit-eligible square may spend its action to build.

Each square has `soil: "good" | "poor"`, authored in level data.

Sequence:

Action	Requirement	Effect
PREPARE	soil is <code>poor</code>	soil becomes <code>good</code> . Takes the piece's whole action.
PLANT	soil is <code>good</code> , square empty of cornerstone	creates a cornerstone with <code>turnsRemaining = 2</code>

The cornerstone ticks down by 1 each UPKEEP phase. At 0 it is `complete` and starts giving light.

While `turnsRemaining > 0` the cornerstone is incomplete: it gives no light, and if a threat moves onto it, the cornerstone is destroyed and must be replanted from scratch.

Soil quality is **hidden** unless the player brought the Knowledge helper. Attempting `PLANT` on poor soil fails and wastes the action — that is how the player finds out.

8. HELPERS (SPIRITS)

The player picks exactly one before each level. **Implement only the first three for v1.** Stub the rest with a "coming soon" state.

id	Name	Effect
counsel	Counsel	Forecast shows 3 turns ahead instead of 2
might	Might	Protector's cover extends to all 8 adjacent squares instead of 4
knowledge	Knowledge	All soil quality visible from turn 1
understanding	Understanding	(stub)
fear	Fear of the Lord	(stub)
wisdom	Wisdom	(stub)
spirit	Spirit of the Lord	(stub)

9. THE FORECAST LAYER

The signature mechanic. Build it early.

- The player toggles between **Now** and **Coming** views with the `Tab` key and with an on-screen button.
- The **Coming** view shows the board as it is, plus markers on squares where threats will spawn within the forecast window, each labelled with the turn number.
- Default window: **2 turns ahead**. With Counsel: 3. Levels 5 and 6 override this down to **1**.

The forecast reads directly from the level's spawn schedule. It is not a simulation and does not predict threat movement — only arrivals.

10. WIN AND LOSE

Win when the level's turn count is completed AND all three house squares are lit. **Lose** when `templeHits >= 3`. Immediate — end the level.

If the turn count completes but the houses are not all lit: level failed, but with a different end screen than losing the temple. Offer retry.

11. TECH STACK

Use exactly this. Do not substitute.

Layer	Choice
Build	Vite
Language	TypeScript, strict: true
UI	React 18
State	Zustand, one store
Styling	CSS Modules
Tests	Vitest
Persistence	localStorage, one JSON blob, versioned
Deploy	Static. Any host.

No canvas. The board is 49 squares. Render it as DOM elements in a CSS grid. Pieces are absolutely positioned divs that transition between squares.

No game engine. No Phaser, no PixiJS, no Godot.

No runtime randomness. If you find yourself typing `Math.random()`, you have misunderstood the design. The only acceptable use is cosmetic particle jitter.

12. FILE LAYOUT

```
src/
  main.tsx
  App.tsx
  game/
    types.ts           # all shared types
    constants.ts       # TEMPLE, HOUSES, BOARD_SIZE, tie-break order
    distance.ts        # Chebyshev helpers
    light.ts           # isLit, litSquares
    threatAI.ts        # pure movement function – heavily tested
    turnEngine.ts      # phase machine: player -> enemy -> upkeep
    destroyer.ts       # overuse counting + auto-fire
    building.ts        # prepare / plant / tick
    rules.ts           # legal move + legal action generation
  store/
    useGame.ts         # Zustand store
    persist.ts         # localStorage, versioned, migration path
  ui/
    Board.tsx
    Square.tsx
    Piece.tsx
    ForecastToggle.tsx
    TurnBar.tsx
    HelperSelect.tsx
    LevelEnd.tsx
  levels/
    level01.ts ... level06.ts
    schema.ts
  styles/
    palette.css        # 24 colours as CSS custom properties
    board.css
```

Level data shape

```
export type Level = {
  id: number;
  turns: number;
  forecastWindow: number; // 2, or 1 for levels 5-6
  startPositions: Record<PieceId, Pos>;
  spawns: Spawn[];
  soil: Record<string, "good" | "poor">; // key: "D6", absent = "good"
};
```

Levels are plain data files. **No level logic in code.** A level must be editable by someone who cannot program.

13. BUILD ORDER — FOLLOW THIS EXACTLY

Do not jump ahead. Each step must work before the next starts.

Step 1 — Board and state. Render a 7×7 grid with the temple and houses. No pieces, no turns. Confirm coordinate display matches (top-left square reads A1, temple reads D4).

Step 2 — Pieces and movement. Place the four pieces. Implement legal-move highlighting and click-to-move for each, respecting their individual movement rules. No enemies yet.

Step 3 — Threat AI. Write `threatAI.ts` as a pure function first, with Vitest tests, before wiring it to the UI. Test cases required: - moves toward the temple by the shortest route - refuses to enter a locked square - refuses to enter a Protector-covered square - attacks the Looser instead of moving through him - attacks the temple when adjacent, then is removed - ties break in N, NE, E, SE, S, SW, W, NW order - given an identical board, produces an identical result 100 times

Step 4 — Turn engine. Wire the three phases together. Add the spawn schedule. The game is now playable and losable.

Step 5 — Light and building. Implement `isLit`, prepare/plant/tick, and the win condition. Level 1 is now completable.

Step 6 — Forecast layer. Tab toggle, markers, turn labels, Counsel override. **Playtest here.** If this mechanic is not interesting, stop and report back before building anything else.

Step 7 — Destroyer overuse. Counting, persistence across levels, auto-fire behaviour, the single end-of-level line.

Step 8 — Helpers. Counsel, Might, Knowledge. Selection screen before each level.

Step 9 — Levels 2-6. Author as data only.

Step 10 — Art pass. Not before this point.

14. VISUAL DIRECTION

Grungy pixel art. Worn, dim, institutional. A map table in a room with one working bulb. **Not cosy, not cute, no rounded corners, no soft shadows.**

- Locked palette of **24 colours max**, all defined as CSS custom properties in `palette.css`. Nothing in the codebase may use a colour literal outside that file.
- Base tones: muted olive, cold grey, rust, nicotine yellow, near-black.
- **The lamp is the only saturated colour on screen.** Everything else is desaturated. This is the one rule that carries the whole look — protect it.
- As `templeHits` rises, reduce global brightness and desaturate further. At 2 hits the board should be genuinely hard to read. The player should feel it before they consciously notice it.
- Typography: one bitmap/pixel font, used at exact multiples of its design size only (16px, 32px) or it renders blurry. Set `image-rendering: pixelated` and disable font smoothing.
- Animation budget is near zero: pieces slide between squares (120ms), darkness creeps in, the lamp flickers. Nothing else moves.

Accessibility floor, without announcing it: full keyboard play (arrows to select a square, Enter to confirm, Tab for forecast), visible focus rings, and `prefers-reduced-motion` respected by cutting transitions to 0ms.

15. ACCEPTANCE CRITERIA

The build is done when all of these are true:

1. A full playthrough of level 1 matches the scripted walkthrough in the design doc, turn for turn.
 2. `threatAI.ts` passes all tests in step 3, including the determinism test.
 3. Playing the same level with the same inputs twice produces byte-identical state.
 4. `avoidableDestroys` persists across levels and triggers auto-fire at 3.
 5. Nothing in the UI reveals the overuse counter before the end-of-level line.
 6. `grep -r "Math.random" src/` returns nothing outside cosmetic effects.
 7. All 6 levels are pure data files with no logic.
 8. The game runs from a static file server with no network requests after load.
 9. Keyboard-only play is possible from level select to level end.
-

16. WHAT NOT TO DO

- Do not add difficulty settings. The levels are the difficulty.
- Do not add a tutorial or tooltips explaining the pieces. Level 1 is the tutorial.
- Do not warn the player about Destroyer overuse. The delayed consequence is the entire design.
- Do not add randomness "for replayability."
- Do not add sound before step 10.
- Do not make the pieces cute.
- If something in this spec seems wrong, stop and ask rather than improvising. Most of it is deliberate.